
第7章 随机化算法

● 学习要点

- 理解产生伪随机数的算法
- 掌握数值随机化算法的设计思想
- 掌握舍伍德算法的设计思想
- 掌握拉斯维加斯算法的设计思想
- 掌握蒙特卡罗算法的设计思想

引言

- 前面几章所讨论的分治、动态规划、贪心法、回溯和分支限界等算法的每一计算步骤都是确定的，本章所讨论的随机化算法允许执行过程中随机选择下一计算步骤。
- 在多数情况下，当算法在执行过程中面临一个选择时，随机性选择常比最优选择省时，因此随机化算法可在很大程度上降低算法复杂性。
- 随机化算法的一个基本特征是对所求解问题的同一实例用同一随机化算法求解两次可能得到完全不同的效果(所需时间或计算结果)。

引言

- 本章将要介绍的随机化算法包括：
 - 数值随机化算法 求解数值问题的近似解，精度随计算时间增加而不断提高。
 - 舍伍德算法 消除算法最坏情况行为与特定实例之间的关联性，并不提高平均性能，也不是刻意避免算法的最坏情况行为。
 - 拉斯维加斯算法 求解问题的正确解，但可能找不到解。
 - 蒙特卡罗算法 求解问题的准确解，但这个解未必正确，且一般情况下无法有效判定正确性。

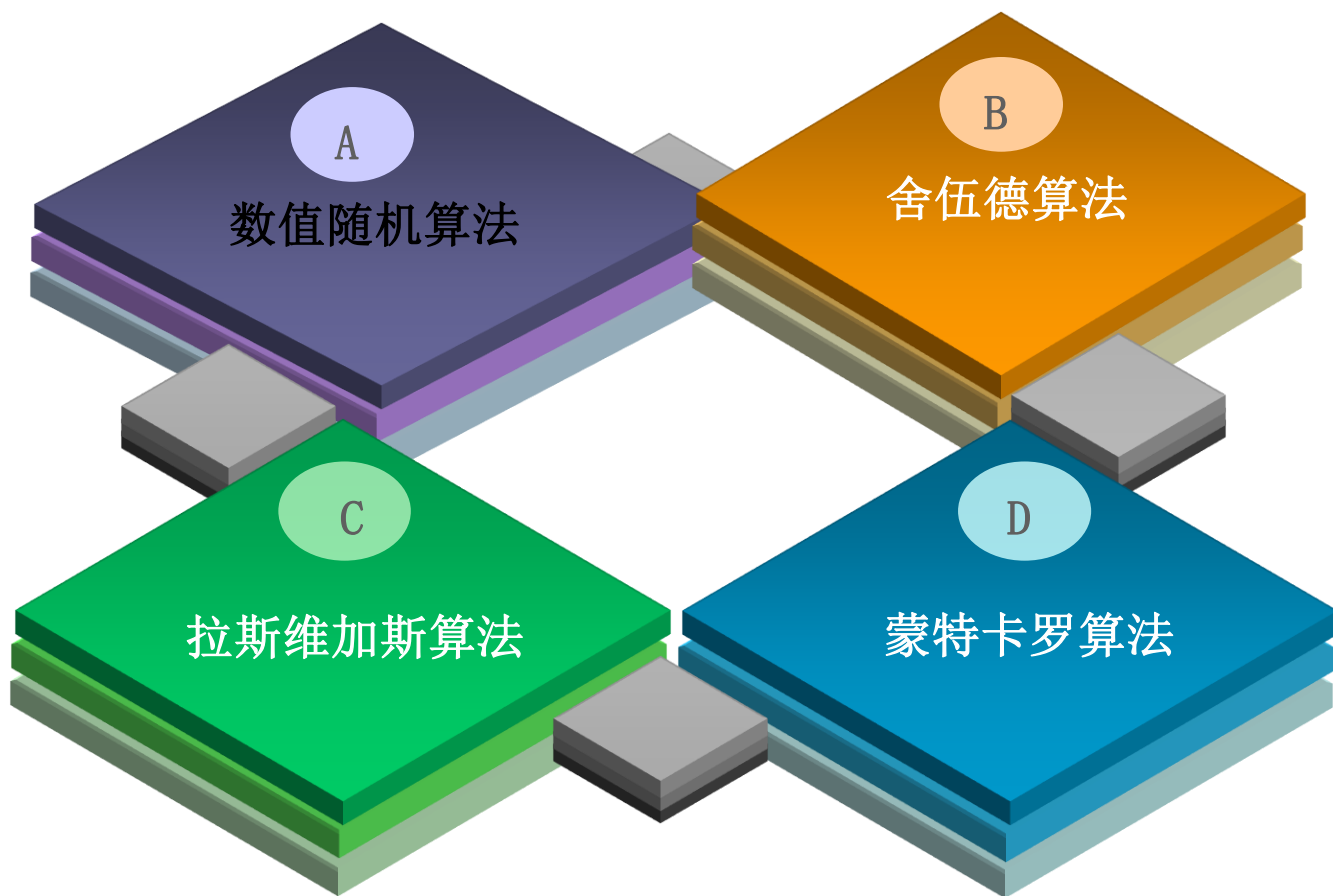
随机化算法概述

● 一个随机化算法 (randomized algorithm) 是指需要利用随机数发生器的算法, 算法执行的某些选择依赖于随机数发生器所产生的随机数。

随机化算法的特点

1. 当算法执行过程中面临选择时，随机化算法通常比最优选择算法省时。
2. 对所求问题的同一实例用同一随机化算法求解两次，两次求解所需的时间甚至所得的结果可能有相当大的差别。
3. 设计思想简单，易于实现。

随机算法分类



下列哪一种算法是随机化（ ）

- ☐ A 贪心算法
- ☐ B 回溯法
- ☐ C 动态规划算法
- ☒ D 舍伍德算法

提交

随机数

随机数在随机化算法设计中扮演着十分重要的角色。

在现实计算机上无法产生真正的随机数，因此在随机化算法中使用的随机数都是一定程度上随机的，即伪随机数。

数值随机算法

数值随机算法 (numerical randomized algorithm)
用于求数值问题的近似解。

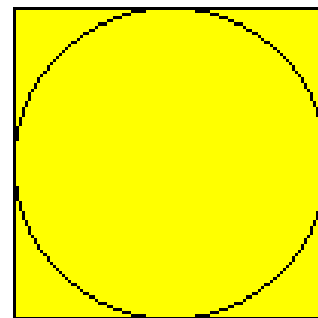
用随机投点法计算 π 值

设有一半径为 r 的圆及其外切四边形。向该正方形随机地投掷 n 个点。设落入圆内的点数为 k 。由于所投入的点在正方形上均匀分布，因而所投入的点落入圆内的概率为：

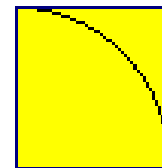
$$\frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

所以当 n 足够大时， k 与 n 之比就逼近这一概率，从而

$$\pi \approx \frac{4k}{n}$$



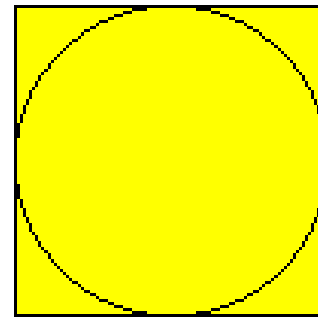
(a)



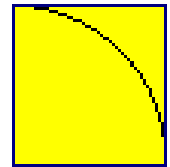
(b)

用随机投点法计算 π 值

```
double Darts(int n)
{ // 用随机投点法计算 $\pi$ 值
  static RandomNumber dart;
  int k=0;
  for (int i=1;i <=n;i++)
  {
    double x=dart.fRandom();
    double y=dart.fRandom(); //产生[0,1)之间的随机实数
    if ((x*x+y*y)<=1) k++;
  }
  return 4*k/double(n);
}
```



(a)



(b)

用随机投点法计算 π 值的运行实例

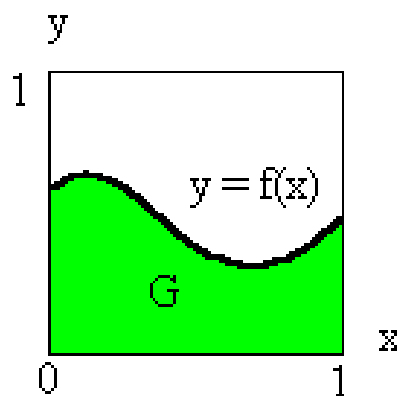
```
随机投掷500个点时, PI=3.128000  
随机投掷5000个点时, PI=3.114400  
随机投掷50000个点时, PI=3.150080  
随机投掷500000个点时, PI=3.138176  
随机投掷5000000个点时, PI=3.142504  
随机投掷50000000个点时, PI=3.141907  
请按任意键继续. . .
```

```
随机投掷500个点时, PI=3.120000  
随机投掷5000个点时, PI=3.105600  
随机投掷50000个点时, PI=3.140640  
随机投掷500000个点时, PI=3.143536  
随机投掷5000000个点时, PI=3.141733  
随机投掷50000000个点时, PI=3.141672  
请按任意键继续. . .
```

计算定积分

设 $f(x)$ 是 $[0, 1]$ 上的连续函数, 且 $0 \leq f(x) \leq 1$ 。

需要计算的积分为 $I = \int_0^1 f(x)dx$, 积分 I 等于图中的绿色面积 G 。



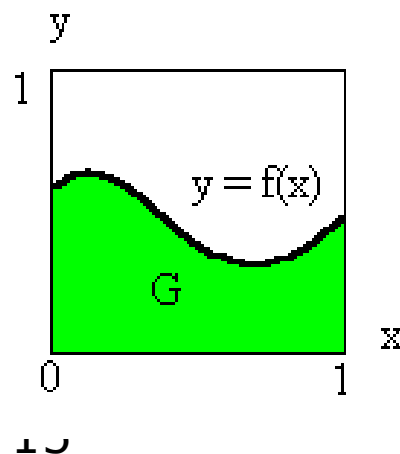
计算定积分

在图所示单位正方形内均匀地作投点试验，则随机点落在曲线 $y = f(x)$ 下面的概率为：

$$P_r \{y \leq f(x)\} = \int_0^1 \int_0^{f(x)} dy dx = \int_0^1 f(x) dx$$

假设向单位正方形内随机地投入 n 个点 (x_i, y_i) 。如果有 m 个点落入 G 内，则随机点落入 G 内的概率：

$$I \approx \frac{m}{n}$$



计算定积分

程序代码如下：

```
double Darts(int n)
{ // 用随机投点法计算定积分
    static RandomNumber dart;
    int k=0;
    for (int i=1;i <=n;i++) {
        double x=dart.fRandom();
        double y=dart.fRandom();
        if (y<=f(x)) k++;
    }
    return k/double(n);}

```


计算定积分的运行实例

```
用随机投点法计算定积分
f(x)=x*x, 范围[0, 1], 真值为1/3
随机投掷100个点时, 定积分=0.330621
随机投掷1000个点时, 定积分=0.327572
随机投掷10000个点时, 定积分=0.337078
随机投掷100000个点时, 定积分=0.332539
随机投掷1000000个点时, 定积分=0.333498
随机投掷10000000个点时, 定积分=0.333212
请按任意键继续. . .
```

```
用随机投点法计算定积分
f(x)=x*x, 范围[0, 1], 真值为1/3
随机投掷100个点时, 定积分=0.341266
随机投掷1000个点时, 定积分=0.324336
随机投掷10000个点时, 定积分=0.330812
随机投掷100000个点时, 定积分=0.333836
随机投掷1000000个点时, 定积分=0.333299
随机投掷10000000个点时, 定积分=0.333279
请按任意键继续. . .
```

实验7-1： 计算定积分(随机化算法)

- 问题描述：用随机投点法计算定积分。

- 示例 1:

输入： $f(x)=x*x$, 区间 $a = 0.0$, $b = 1.0$ 投点次数 $n=100$

输出： 0.330632

- 要求:

- 1 填空完成程序;

- 2 输出不同投点次数时定积分的值。

- *3 改变函数表达式、区间值为 $f(x)=1/3*x*x*x$, 区间 $a = 1.0$, $b = 3.0$, 计算定积分。

舍伍德(Sherwood)算法

设A是一个确定性算法，当它的输入实例为 x 时所需的计算时间记为 $t_A(x)$ 。设 X_n 是算法A的输入规模为 n 的实例的全体，则当问题的输入规模为 n 时，算法A所需的平均时间为：

$$\bar{t}_A(n) = \sum_{x \in X_n} t_A(x) / |X_n|$$

舍伍德(Sherwood)算法

这显然不能排除存在 $x \in X_n$ 使得 $t_A(x) \gg \bar{t}_A(n)$ 的可能性。希望获得一个随机化算法B，使得对问题的输入规模为 n 的每一个实例均有：

$$t_B(x) = \bar{t}_A(n) + s(n)$$

这就是舍伍德算法设计的基本思想。

当 $s(n)$ 与 $\bar{t}_A(n)$ 相比可忽略时，舍伍德算法可获得很好的平均性能。

舍伍德(Sherwood)算法

▪ No.1

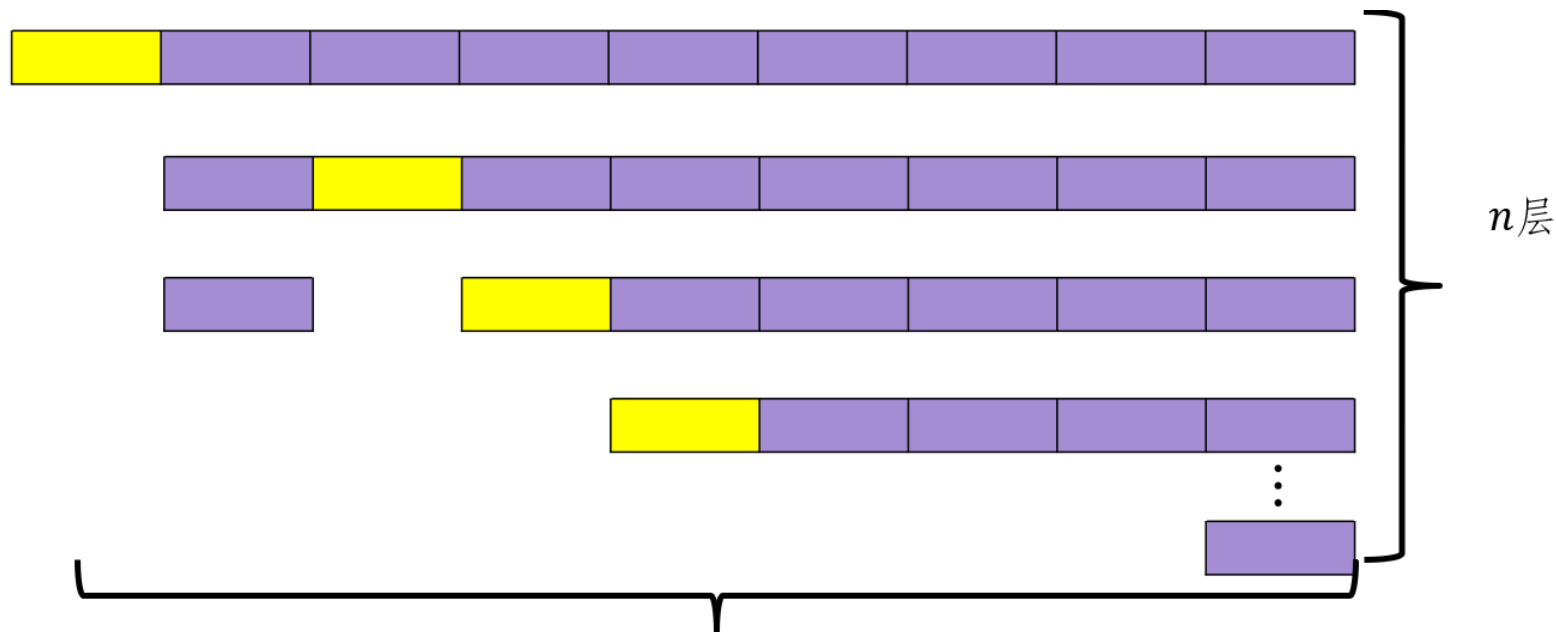
线性时间选择算法

▪ No.2

快速排序算法

这两种算法的核心都在于选择合适的划分基准。舍伍德算法随机地选择一个数组元素作为划分基准。

快速排序算法中pivot（基准）选择



剩余 i 个元素时为 $O(i)$

$$T(n) = T(0) + T(n-1) + cn$$

$$\Theta\left(\sum_{k=1}^n k\right) = \Theta(n^2)$$

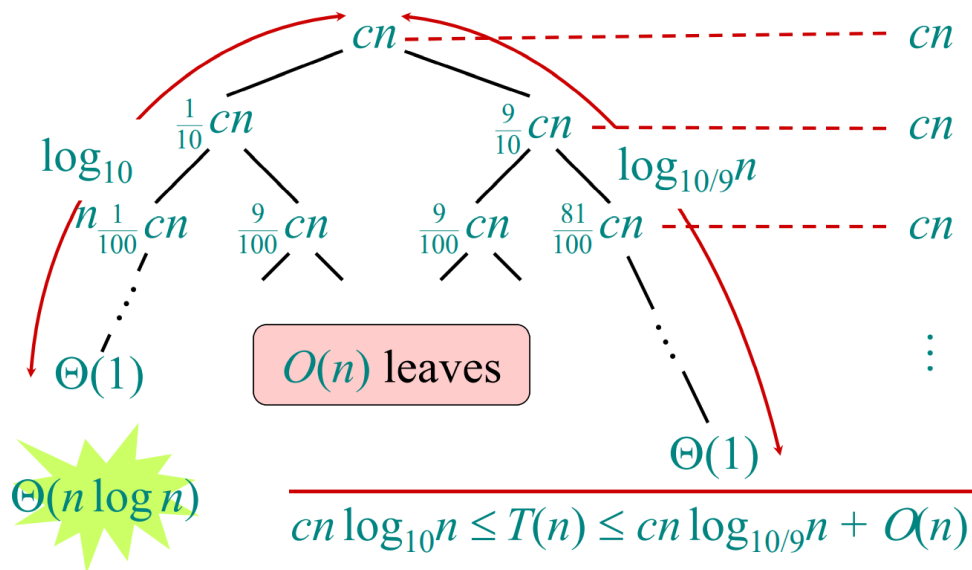
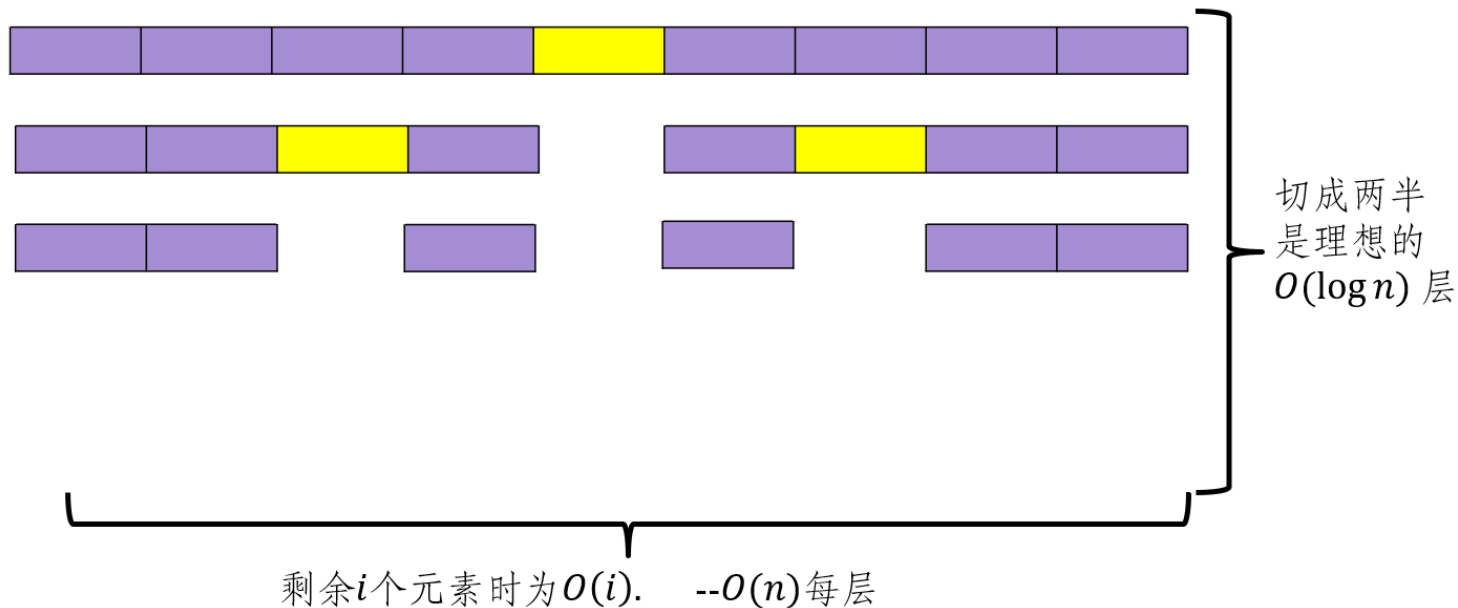
$$T(n) = \Theta(n) + \Theta(n^2) = \Theta(n^2)$$

height = n

The diagram shows a recursion tree for the worst-case scenario of quicksort. The root node is labeled cn . It has two children: $\Theta(1)$ on the left and $c(n-1)$ on the right. The $c(n-1)$ node has two children: $\Theta(1)$ on the left and $c(n-2)$ on the right. This pattern continues down to a single $\Theta(1)$ node at the bottom. A yellow diagonal band highlights the path of the recursive calls. A red arrow points from the summation formula to the $c(n-1)$ node. A vertical red double-headed arrow on the left indicates the height of the tree is n .

分析最差情况

快速排序算法中pivot选择



分析“几乎最好”情况

常见的快速排序实现

实践中的一个常见策略是“**三的中位数**”规则。

选择三个元素（随机或从特定位置）。取其中的中位数作为pivot，保证没有最差的pivot。

在固定pivot之外只有一小部分恒定数量的额外步骤（找到三个数字的中位数只是一些比较）。

另一种策略：找到真正的中位数（不切实际：取421）

舍伍德(Sherwood)算法 快速排序

```
int Partition(int a[], int low, int high)
{
    int key = a[low];
    int i = low, j = high + 1;
    while (1)
    {
        while (a[++i] < key && i <= high);
        while (a[--j] > key && j >= low);
        if (i >= j) break;
        if (a[i] != a[j])
            swap(a[i], a[j]);
    }
    if ((j != low) && (a[low] != a[j]))
        swap(a[low], a[j]);
    return j;
}
```

```
int Randpartition(int a[], int low, int high)
{
    int k;
    srand(time(NULL));
    //随机生成一个low,high之间的位置k
    k = rand() % (high - low + 1) + low;
    //将a[k]作为pivot
    swap(a[k], a[low]);
    k = Partition(a, low, high);
    return k;
}
```

```
void Quick_sort(int a[], int low, int high)
{
    if (low < high)
    {
        int k = RandPartition(a, low, high);
        Quick_sort(a, low, k - 1);
        Quick_sort(a, k + 1, high);
    }
}
```

舍伍德(Sherwood)算法 快速排序实例

第一次洗牌:

19 8 7 14 20 16 11 17 1 15 6 13 10 2 3 12 4 9 5 18

第二次洗牌:

5 17 11 2 18 12 6 4 19 3 16 10 15 8 7 13 14 1 20 9

选择的pivot位置, 值序列为:

17:1, 14:7, 3:2, 3:6, 2:4, 19:9, 18:20, 16:14, 10:11
, 12:12, 16:15, 16:17, 18:18,

排序后: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
19 20

请按任意键继续. . .

实验7-2：舍伍德快速排序(随机化算法)

- 问题描述: 利用舍伍德算法实现快速排序。
 - 示例 1:
输入100数值。
 - 要求:
 - 1 填空完成程序;
 - 2 输出排序结果。
 - *3 设计一个划分算法, 利用3个随机数的中位数作为pivot。
-

舍伍德(Sherwood)算法

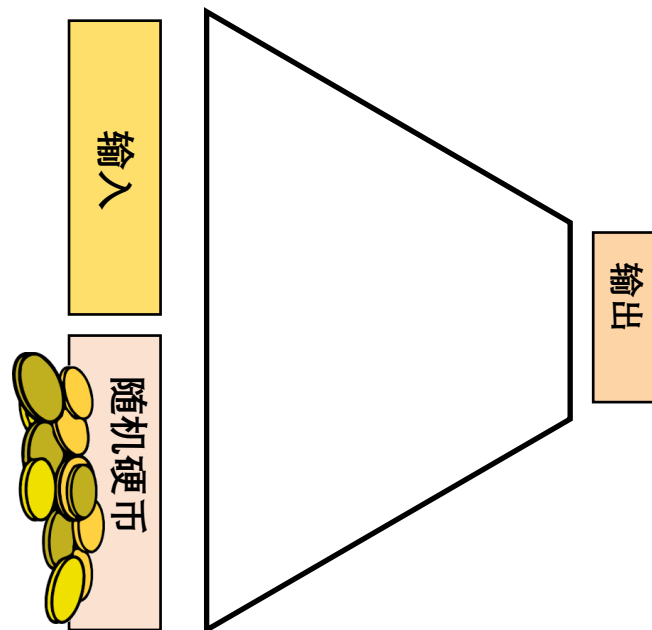
有时也会遇到这样的情况，即所给的确定性算法无法直接改造成舍伍德型算法。

此时可借助于随机预处理技术，不改变原有的确定性算法，仅对其输入进行随机洗牌，同样可收到舍伍德算法的效果。

两个等效视图：



1. 获取输入 $x \in \{0,1\}^n$
2. 运行算法 $A(x)$ 具有特殊运算
 $r_i \leftarrow \text{RAND}()$
($r_i \sim \{0,1\}$)



1. 获取输入 $x \in \{0,1\}^n$
2. 选择 $r \sim \{0,1\}^m$
3. 运行确定性算法 $A(x, r)$

$$\textit{output} = \textit{ALG}(\textit{input}, \textit{randomness})$$

舍伍德(Sherwood)算法

例如，对于确定性选择算法，可以用下面的洗牌算法 shuffle 将数组a中元素随机排列，然后用确定性选择算法求解。这样做所收到的效果与舍伍德型算法的效果是一样的。

```
template<class Type>
void Shuffle(Type a[], int n)
{ // 随机洗牌算法
    static RandomNumber rnd;
    for (int i=0; i<n; i++)
    {
        int j=rnd.Random(n-i)+i;
        Swap(a[i], a[j]);
    }
}
```

随机洗牌算法的运行实例

原次序:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78
79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```

第一次洗牌:

```
19 50 82 70 46 86 16 8 89 65 32 36 21 74 78 72 17 15 77 13 62 96 12 48 9 87
84 7 39 29 64 92 49 1 47 55 67 85 75 34 90 56 83 3 93 99 30 63 60 28 88 4
25 42 45 80 73 91 97 31 100 61 53 76 98 6 10 22 94 5 69 20 27 37 52 68 33 5
4 40 59 35 14 57 43 41 44 26 51 18 79 24 2 66 38 81 11 58 71 23 95
```

第二次洗牌:

```
77 28 14 5 99 44 72 8 18 98 92 55 62 37 54 20 17 78 33 21 61 11 36 63 89 26
43 16 75 39 76 2 60 19 30 45 10 41 52 1 79 80 57 82 66 23 29 53 31 7 51 70
9 56 93 59 27 24 58 64 95 100 25 68 71 86 65 96 38 46 94 13 84 67 4 22 49
42 34 97 47 74 73 83 90 3 87 88 15 40 48 50 6 85 35 32 91 69 12 81
```

舍伍德算法

总能求得问题的正确解。当一个确定性算法在最坏情况下的计算复杂度与其在平均情况下的计算复杂度两者相差较大时，可以在这个确定算法中引入随机性将它改造成一个舍伍德算法，用来消除或减少问题的不同实例之间在计算时间上的差别。舍伍德算法的精髓不是避免算法的最坏情况行为，而是设法消除这种最坏行为与特定实例之间的关联性。

拉斯维加斯(Las Vegas)算法

拉斯维加斯算法的一个显著特征是它所作的随机性决策有可能导致算法找不到所需的解。

```
void obstinate(Object x, Object y)
{
    // 反复调用拉斯维加斯算法LV(x,y),
    // 直到找到问题的一个解y
    bool success= false;
    while (!success) success=lv(x,y);
}
```

拉斯维加斯(Las Vegas)算法

设 $p(x)$ 是对输入 x 调用拉斯维加斯算法获得问题的一个解的概率。一个正确的拉斯维加斯算法应该对所有输入 x 均有 $p(x) > 0$ 。设 $t(x)$ 是算法obstinate找到具体实例 x 的一个解所需的平均时间， $s(x)$ 和 $e(x)$ 分别是算法对于具体实例 x 求解成功或求解失败所需的平均时间，则有：

$$t(x) = p(x)s(x) + (1 - p(x))(e(x) + t(x))$$

解此方程可得：

$$t(x) = s(x) + \frac{1 - p(x)}{p(x)} e(x)$$

n后问题的拉斯维加斯算法

对于n后问题的任何一个解而言，每一个皇后在棋盘上的位置无任何规律，不具有系统性，而更象是随机放置的。由此容易想到拉斯维加斯算法。

在棋盘上相继的各行中随机地放置皇后，并注意使新放置的皇后与已放置的皇后互不攻击，直至n个皇后均已相容地放置好，或已没有下一个皇后的可放置位置时为止。

n后问题

如果将上述随机放置策略与回溯法相结合，可能会获得更好的效果。

可以先在棋盘的若干行中随机地放置皇后，然后在后继行中用回溯法继续放置，直至找到一个解或宣告失败。

随机放置的皇后越多，后继回溯搜索所需的时间就越少，但失败的概率也就越大。

n后问题的拉斯维加斯算法

```
bool Queen::QueensLV(int stop){//随机放置n个皇后的拉斯维斯加算法
    RandomNumber rnd;    // 随机数产生器
    int k = 1;            //下一个放置的皇后的编号
    int count = 1;
    while ((k <= stop) && (count>0)){
        count = 0;
        for (int i = 1; i <= n; i++){//遍历第k行所有可以放置的列号，用y保存，并用count记录有多少个位置可以放置
            x[k] = i;
            if (Place(k))
                y[count++] = i;
        }
        if (count>0)    //利用上面保存的可以放置的列，然后随机取其中一列来放置第k行的皇后。是Las Vegas思想
            x[k++] = y[rnd.Random(count)];//随机位置
    }
    return (count>0); //count>0表示放置成功
}
```

n后问题

允许随机放置的皇后数

n=12

stopVegas	p	s	e	t
0	1.0000	262.00	--	262.00
5	0.5039	33.88	47.23	80.39
12	0.0465	13.00	10.20	222.11

p: 成功概率

s: 成功搜索访问的节点数平均值

e: 不成功搜索访问的节点数平均值

t: 找到一个解访问的节点数平均值

n后问题的运行实例

12皇后问题的解为（随机放置的皇后数为5）：
3 1 9 7 10 4 2 12 5 11 8 6
请按任意键继续. . .

12皇后问题的解为（随机放置的皇后数为5）：

无解
请按任意键继续. . .

12皇后问题的解为（随机放置的皇后数为5）：
5 9 12 4 2 8 6 1 3 11 7 10
请按任意键继续. . .

实验7-3：拉斯维加斯算法实现nQueen

- 问题描述：nQueen随机放置位置策略和回溯法相结合求解n皇后问题。拉斯维加斯算法不会得到不正确的解。一旦用拉斯维加斯算法找到一个解，这个解就一定是正确解。但有时用拉斯维加斯算法找不到解。与蒙特卡罗算法类似，拉斯维加斯算法找到正确解的概率随着它所用的计算时间的增加而提高。
- 示例 1：皇后数 $n=12$ 随机放置皇后数 $\text{stop}=5$
- 要求：
 - 1 填空完成程序；
 - 2 输出放置结果，运行次数。
 - *3 改变皇后数与随机放置皇后数，输出结果。

拉斯维加斯算法

求得的解总是正确的，但有时拉斯维加斯算法可能始终找不到解。一般情况下，求得正确解的概率随计算时间的增加而增大。因此，为了减少求解失败的概率，可以使用一个拉斯维加斯算法对同一实例，重复多次执行该算法。

蒙特卡罗(Monte Carlo)算法

- 在实际应用中常会遇到一些问题，不论采用确定性算法或随机化算法都无法保证每次都能得到正确的解答。蒙特卡罗算法则在一般情况下可以保证对问题的所有实例都以高概率给出正确解，但是通常无法判定一个具体解是否正确。
- 设 p 是一个实数，且 $1/2 < p < 1$ 。如果一个蒙特卡罗算法对于问题的任一实例得到正确解的概率不小于 p ，则称该蒙特卡罗算法是 p 正确的，且称 $p - 1/2$ 是该算法的优势。

蒙特卡罗(Monte Carlo)算法

- 如果对于同一实例，蒙特卡罗算法不会给出2个不同的正确解答，则称该蒙特卡罗算法是一致的。
- 有些蒙特卡罗算法除了具有描述问题实例的输入参数外，还具有描述错误解可接受概率的参数。这类算法的计算时间复杂性通常由问题的实例规模以及错误解可接受概率的函数来描述。

蒙特卡罗(Monte Carlo)算法

对于一个一致的p正确蒙特卡罗算法，要提高获得正确解的概率，只要执行该算法若干次，并选择出现频次最高的解即可。

如果重复调用一个一致的 $(1/2+\varepsilon)$ 正确的蒙特卡罗算法 $2m-1$ 次 ($m = \lfloor n/2 \rfloor + 1$)，得到正确解的概率至少为 $1-\delta$ ，其中，

$$\delta = \frac{1}{2} - \varepsilon \sum_{i=0}^{m-1} \binom{2i}{i} \left(\frac{1}{4} - \varepsilon^2\right)^i \leq \frac{(1-4\varepsilon^2)^m}{4\varepsilon\sqrt{\pi m}}$$

蒙特卡罗(Monte Carlo)算法

对于一个解所给问题的蒙特卡罗算法 $MC(x)$ ，如果存在问题实例的子集 X 使得：

(1) 当 $x \notin X$ 时， $MC(x)$ 返回的解是正确的；

(2) 当 $x \in X$ 时，正确解是 y_0 ，但 $MC(x)$ 返回的解未必是 y_0 。

称上述算法 $MC(x)$ 是偏 y_0 的算法。

重复调用一个一致的， p 正确偏 y_0 蒙特卡罗算法 k 次，可得到一个 $O(1-(1-p)^k)$ 正确的蒙特卡罗算法，且所得算法仍是一个一致的偏 y_0 蒙特卡罗算法。

主元素问题

设 $T[1:n]$ 是一个含有 n 个元素的数组。当 $|\{i|T[i]=x\}| > n/2$ 时，称元素 x 是数组 T 的主元素。

```
template<class Type>
bool Majority(Type *T, int n)
{// 判定主元素的蒙特卡罗算法
    int i=rnd.Random(n)+1;
    Type x=T[i];// 随机选择数组元素
    int k=0;
    for (int j=1;j<=n;j++)
        if (T[j]==x) k++;
    return (k>n/2);
    // k>n/2 时T含有主元素
}
```

```
template<class Type>
bool MajorityMC(Type *T, int n, double e)
{// 重复调用k次Majority算法
    int k=ceil(log(1/e)/log(2));
    for (int i=1;i<=k;i++)
        if (Majority(T,n)) return true;
    return false;
}
```

1	2	3	4	3	4	1	2
---	---	---	---	---	---	---	---

1	3	4	3	5	3	3	3
---	---	---	---	---	---	---	---

确定性的算法：

- ◆ 排序、一遍扫描
- ◆ 分治法也可以

主元素问题

对于任何给定的 $\varepsilon > 0$ ，算法MajorityMC重复调用 $\lceil \log(1/\varepsilon) \rceil$ 次算法Majority。它是一个偏真蒙特卡罗算法，且其错误概率小于 ε 。算法MajorityMC所需的计算时间显然是 $O(n \log(1/\varepsilon))$ 。

主元素问题的运行实例

```
int main()
{
    int arr[10] = {3, 5, 7, 3, 9, 3, 3, 1, 3, 3};
    for (int var=0; var<10; var++)
        cout << boolalpha << MajorityMC(arr, 10, 0.1) << endl;
    return 0;
}
```

main重复调用10次MajorityMC, 而MajorityMC 重复调用k次Majority算法

本例中int k=ceil(log(1/e)/log(2)); //e=0.1, k=4

主元素问题的运行实例

```
数组的元素如下:  
3 5 7 3 9 3 3 1 3 3  
true  
true  
true  
true  
true  
true  
false  
true  
true  
true  
请按任意键继续. . .
```

主元素问题的运行实例

```
数组的元素如下:  
3 5 7 3 9 3 3 1 3 3  
true  
true  
true  
true  
true  
true  
true  
true  
true  
true  
true  
请按任意键继续. . .
```

主元素问题的运行实例

```
数组的元素如下:  
3 5 7 3 9 3 3 1 3 3  
false  
true  
true  
true  
true  
true  
true  
true  
false  
true  
true  
请按任意键继续. . .
```

搜索树结点数估计

Monte Carlo方法

1. 从根开始，随机选择一条路径，直到不能分支为止，即从 $x_1, x_2, \dots$ ，依次对 x_i 赋值，每个 x_i 的值是从当时的 S_i 中随机选取，直到向量不能扩张为止。
 2. 假定搜索树的其他 $|S_i| - 1$ 个分支与以上随机选出的路径一样，计数搜索树的点数。
 3. 重复步骤 1 和 2，将结点数进行概率平均。
-

伪码

Monte Carlo

输入： n 为皇后数， t 为抽样次数

输出： sum , 即 t 次抽样路长平均值

1. $sum \leftarrow 0$
2. for $i \leftarrow 1$ to t do // 取样次数 t
3. $m \leftarrow \text{Estimate}(n)$ // m 为结点数
4. $sum \leftarrow sum + m$
5. $sum \leftarrow sum / t$



一次抽
样结果

一次抽样

m 为本次取样得到的树结点总数

k 为层数

r_2 为上层结点数

r_1 为本层结点数

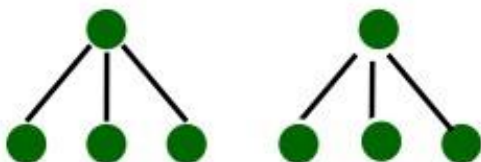
$r_1 = r_2 \cdot \text{分支数}$

n 为树的层数

从树根向下计算,随机选择,直到树叶.

$$r_2 = 2$$

$$r_1 = r_2 \cdot 3 = 6$$



子过程的伪码

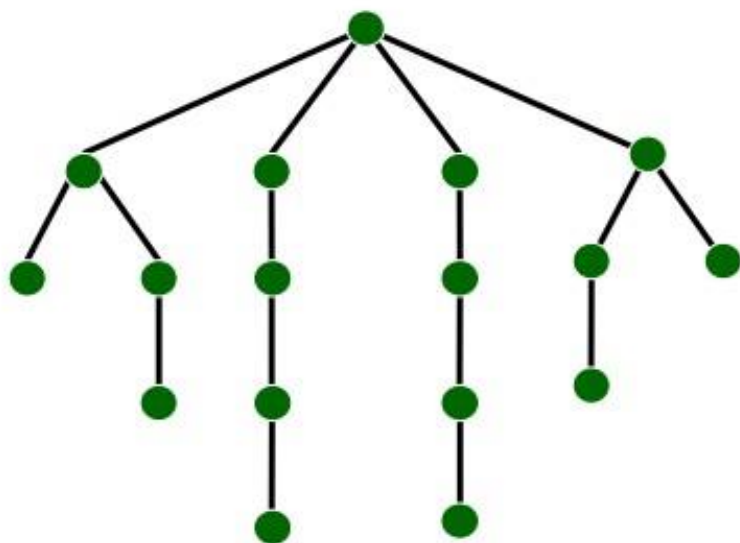
算法Estimate(n)

1. $m \leftarrow 1$; $r_2 \leftarrow 1$; $k \leftarrow 1$ // m 为结点总数
2. while $k \leq n$ do
3. if $S_k = \emptyset$ then return m
4. $r_1 \leftarrow |S_k| * r_2$ // r_1 为扩张后结点总数
5. $m \leftarrow m + r_1$ // r_2 为扩张前结点总数
6. $x_k \leftarrow$ 随机选择 S_k 的元素
7. $r_2 \leftarrow r_1$
8. $k \leftarrow k + 1$

不能
分支

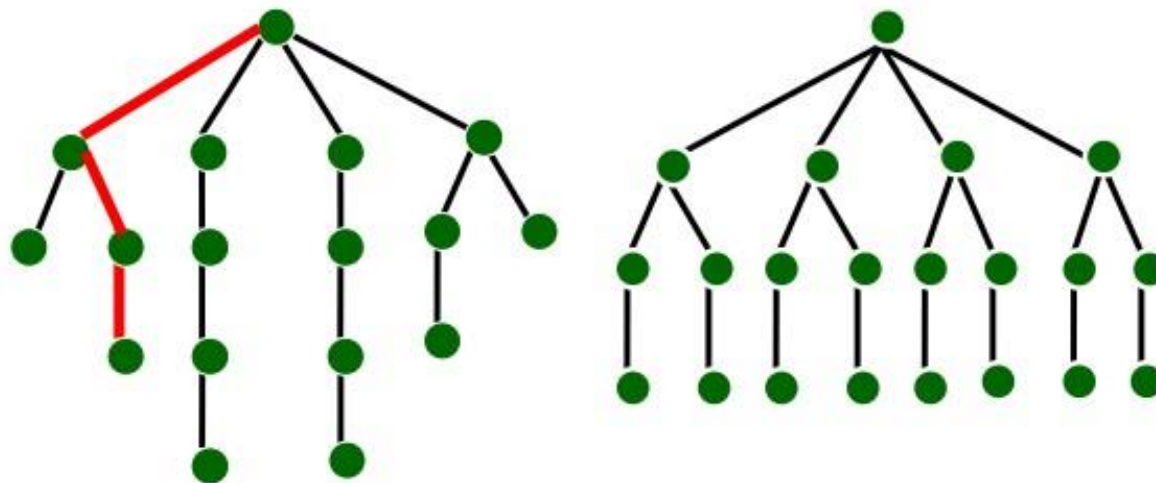
随机选
择一步

4后搜索树遍历的结点



结点数=17

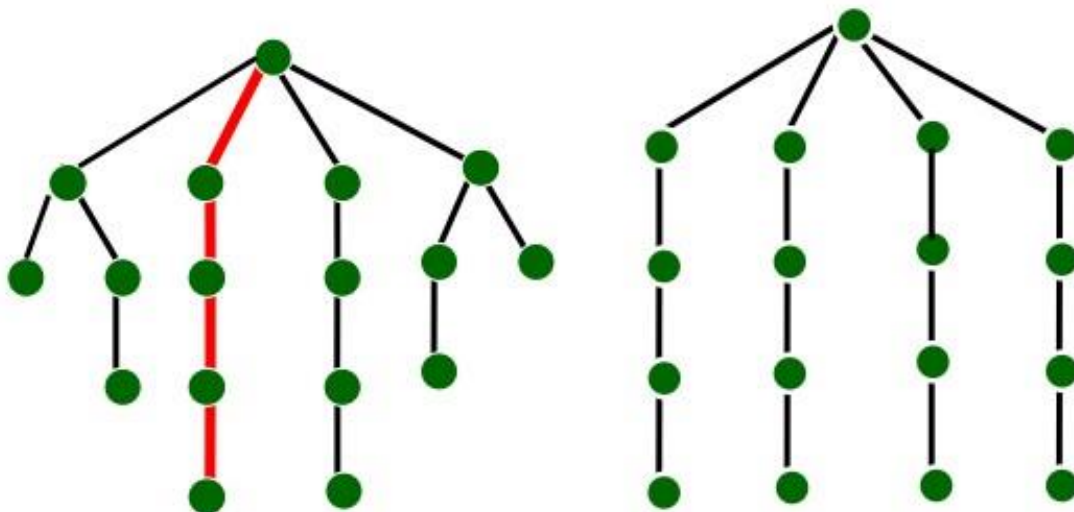
随机选择路径1



Case1: $\langle 1, 4, 2 \rangle$,

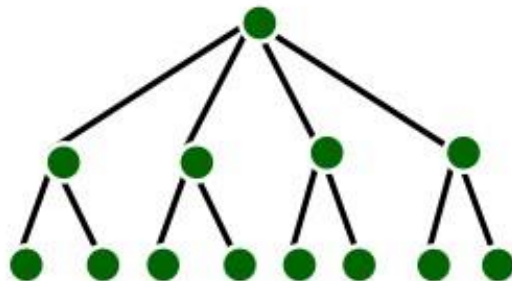
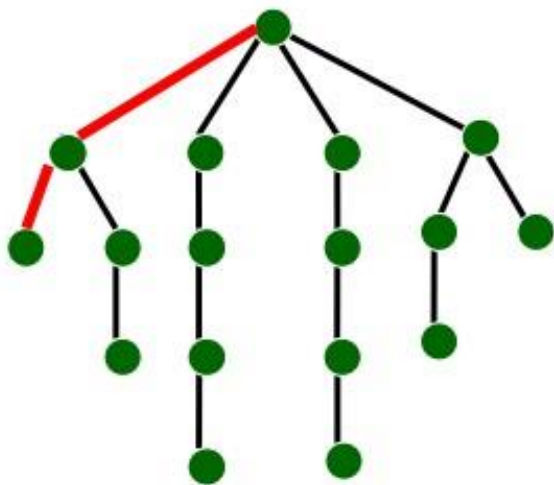
结点数 21

随机选择路径2



Case1: $\langle 2, 4, 1, 3 \rangle$, 结点数 17

随机选择路径3



Case3: $\langle 1, 3 \rangle$,

结点数 13

估计结果

假设 4 次抽样测试:

case1: 1次,

case2: 1次,

case3: 2次,

平均结点数 $= (21 \times 1 + 17 \times 1 + 13 \times 2) / 4 = 16$

搜索空间访问的结点数为 17

小结

- **Monte Carlo 方法**

目的： 估计搜索树真正访问结点数

步骤：

随机抽样，选择一条路径

用这条路径代替其他路径

逐层累加树的结点数

多次选择，取结点数的平均值

蒙特卡罗算法

该法用于求问题的准确解(或者说是精确解而不是近似解)，但不保证所求得的解是正确的，也就是说，蒙特卡罗算法求得的解有时是错误的。不过，由于可以设法控制这类算法得到错误解的概率，并因它的简单高效，是很有价值的一类随机算法。

● 总结

随机化算法的基本特征就是对所求解问题的同一实例用同一随机化算法求解两次可能得到完全不同的效果。

算法	定义	实例
数值概率算法	求数值问题的近似解	用投点法求 π
舍伍德算法	必能求得一个正确解。它用来消除输入数据与最坏情况的关联性。	随机化快速排序
拉斯维加斯算法	找到的解必为正确解，但有时找不到解	n后问题
蒙特卡罗算法	必能求得一个解，但解不一定正确	主元素问题

在下列算法中有时找不到问题解的是（ ）。

- ☐ A 蒙特卡罗算法
- ☒ B 拉斯维加算法
- ☐ C 舍伍德算法
- ☐ D 数值概率算

提交